LAB SHEET 07

Django Templates – Fruits & Selected Event Students

Primary Syllabus Exercise Reference

Develop a complete Django app mapping model array collections out to unordered item matrices (Fruits) alongside ordered indices (Selected Event Students).

Task 7.1 – Fallback Condition Logic

Implement robust fallback condition logic filters inside the Django template layer ({% if %}) to show an alert if arrays arrive empty.

views.py

```
from django.shortcuts import render

def index(request):

        fruits = ["Apple", "Banana", "Mango", "Orange"]

 students = [

    {"name": "Aman", "event": "Coding"},

    {"name": "Riya", "event": "Quiz"},

    {"name": "Karan", "event": "Hackathon"},

]

 return render(request, "students/index.html", {

    "fruits": fruits,

    "students": students

 })
```

index.html

```
{% if fruits %}

    <h3>Fruits</h3>

    <ul>

      {% for fruit in fruits %}

          <li>{{ fruit }}</li>

      {% endfor %}

      </ul>
```

```
{% else %}
        <div class="alert">No fruits are available.</div>
{% endif %}
{% if students %}
        <h3>Selected Event Students</h3>
{% else %}
        <div class="alert">No students were selected.</div>
{% endif %}
```

Task 8.2 – Dynamic Table Sorting

Incorporate dynamic table sorting tools inside the template views to order records based on specific field properties, such as student name.

views.py

```
def index(request):
    students = [
        {"name": "Aman", "event": "Coding"},
        {"name": "Riya", "event": "Quiz"},
        {"name": "Karan", "event": "Hackathon"},
]
 sort = request.GET.get("sort", "name")
 if sort == "name_desc":
        students = sorted(students, key=lambda x: x["name"], reverse=True)
 else:
        students = sorted(students, key=lambda x: x["name"])
 return render(request, "students/index.html", {
      "students": students
 })
```

Sorting Controls in Template

```
<a href="?sort=name">Sort Name A–Z</a>
```

```
<a href="?sort=name_desc">Sort Name Z–A</a>
<table border="1">
    <tr>
        <th>Student Name</th>
        <th>Selected Event</th>
    </tr>
    {% for student in students %}
    <tr>
        <td>{{ student.name }}</td>
        <td>{{ student.event }}</td>
    </tr>
    {% endfor %}
</table>
```

Task 8.3 – User Search Input Form

Build a simple user search input form that screens long array matrices and subsets lists dynamically using back-end request processing rules.

Search Logic – views.py

```
def index(request):
        students = [
            {"name": "Aman", "event": "Coding"},
            {"name": "Riya", "event": "Quiz"},
            {"name": "Karan", "event": "Hackathon"},
            {"name": "Neha", "event": "Coding"},
            {"name": "Rahul", "event": "Quiz"},
]
 query = request.GET.get("q", "").strip()
 if query:
     students = [
        student for student in students
```

```
        if query.lower() in student["name"].lower()
    ]
students = sorted(students, key=lambda x: x["name"])
return render(request, "students/index.html", {
        "students": students,
        "query": query
})
```

Search Form – index.html

```
<form method="get">
      <input type="text" name="q"
             placeholder="Search student"
             value="{{ query }}">
    <button type="submit">Search</button>
</form>
{% if students %}
       {% for student in students %}
             <p>{{ student.name }} - {{ student.event }}</p>
      {% endfor %}
{% else %}
        <p>No students found for "{{ query }}".</p>
{% endif %}
```

LAB SHEET 08

Template Inheritance Blueprints & Modular UI Engineering

Primary Syllabus Exercise Reference

Develop a core master template layout blueprint file containing shared menus and block overrides; inherit properties down across specialized views (Home, About Us, Contact Us).

Task 8.1 – Dynamic Active Navigation Highlight

Embed custom programmatic markers within the master view framework to dynamically assign context-specific CSS structural active utility highlights to the active navigation item.

base.html – Master Template

```
<!DOCTYPE html>
<html>
<head>
        <title>{% block title %}My Django Site{% endblock %}</title>
        <style>
            .active { font-weight: bold; text-decoration: underline; }
        </style>
</head>
<body>
    <nav>
        <a class="{% if active_page == 'home' %}active{% endif %}"
           href="/">Home</a>
       <a class="{% if active_page == 'about' %}active{% endif %}"
          href="/about/">About Us</a>
      <a class="{% if active_page == 'contact' %}active{% endif %}"
         href="/contact/">Contact Us</a>
   </nav>
    {% block messages %}{% endblock %}
    {% block content %}{% endblock %}
</body>
</html>
```

views.py

```
def home(request):
    return render(request, "home.html", {"active_page": "home"})
def about(request):
    return render(request, "about.html", {"active_page": "about"})
def contact(request):
    return render(request, "contact.html", {"active_page": "contact"})
```

Task 8.2 – Persistent User Message Feedback

Integrate a persistent user message feedback notification box using inherited template placeholder hooks across all child application page screens.

base.html

```
{% if messages %}
<div class="messages">
    {% for message in messages %}
        <div class="message">{{ message }}</div>
    {% endfor %}
</div>
{% endif %}
{% block content %}{% endblock %}
```

settings.py

```
INSTALLED_APPS = [
    . . .
   "django.contrib.messages",
]
MIDDLEWARE = [
   . . .
   "django.contrib.messages.middleware.MessageMiddleware",
]
```

Example Child Template – home.html

```
{% extends "base.html" %}

{% block title %}Home{% endblock %}

{% block content %}

<h1>Welcome to Home Page</h1>

<p>This page inherits the common navigation and message area.</p>

{% endblock %}
```

Task 8.3 – Professional Contact Us Form

Build a Contact Us form view page that receives feedback text and forwards verified feedback parameters directly into the server logs for system verification.

contact.html

```
{% extends "base.html" %}

{% block title %}Contact Us{% endblock %}

{% block content %}

<h2>Contact Us</h2>

<form method="post">

    {% csrf_token %}

    <label>Name:</label>

    <input type="text" name="name" required>

    <label>Message:</label>

    <textarea name="message" required></textarea>

    <button type="submit">Send Feedback</button>

</form>

{% endblock %}
```

views.py – Form Processing & Server Log

```
import logging

from django.contrib import messages

from django.shortcuts import render, redirect
```

```
logger = logging.getLogger(__name__)
def contact(request):
    if request.method == "POST":
        name = request.POST.get("name", "").strip()
        message = request.POST.get("message", "").strip()
        if name and message:
            logger.info("Contact feedback received from %s: %s", name, message)
            messages.success(request, "Your feedback has been submitted.")
            return redirect("contact")
    return render(request, "contact.html",
        {"active_page": "contact"})
```

Output / Working

1. Home, About Us and Contact Us pages inherit the common base.html layout.
2. The current navigation item receives the active CSS class.
3. Django messages appear in the common notification area.
4. The Contact Us form accepts user feedback and logs valid submitted data on the server.